Smart Placement Portal – CI/CD Pipeline (CCA 2)

Automated CI/CD pipeline for the **Smart Placement Portal**, an existing academic project, using GitHub, Jenkins on AWS, Docker, Docker Hub and Kubernetes (Minikube).

- **GitHub repo:** https://github.com/Tamanna-Roy0812/smart_placement (branch: `frontend-raashi`)
- **Docker Hub image:** `tamannaroy/smart-placement`
- **Name / PRN / Division:** Tamanna Roy | PRN 1262243558 | Division G

1. Project Chosen

Smart Placement Portal is a smart internship and placement portal where students can upload their resume, take assessments and check whether they are eligible for available jobs. Tech stack: React (Vite) frontend served by Nginx in Docker. I chose it because it is a complete working project that builds and runs without manual fixes.

2. Repository Structure

```
.
├── backend/
├── frontend/   (React + Vite, nginx.conf, Dockerfile)
├── Dockerfile
├── Jenkinsfile
└── k8s/
    ├── deployment.yaml
    └── service.yaml
```

3. Architecture

GitHub → Jenkins (AWS EC2) → Docker build → Docker Hub → Minikube (Deployment + Service) → Browser

4. Pipeline Stages (Jenkinsfile)

#	Stage	What it does

1	Checkout	Jenkins pulls the latest code from GitHub
2	Build and Test	Runs `npm install` and `npm run build` inside the `frontend` folder. A successful Vite production build acts as the test step, since it fails on syntax or import errors
3	Docker Build	Builds the image from the Dockerfile and tags it with the build number and `latest`
4	Push Image	Logs in to Docker Hub using Jenkins credentials and pushes both tags
5	Minikube Deployment	Applies `k8s/deployment.yaml` and `k8s/service.yaml` with kubectl, updating the image to the new tag
6	Verification	Runs `kubectl get pods,deployment,svc` and waits for the rollout to finish

5. Jenkins Setup on AWS

1. Launched an AWS EC2 instance (Ubuntu, about 4 GB RAM) and opened ports 22, 8080 (Jenkins) and the app NodePort in the security group.
2. Installed Java, Jenkins, Docker, kubectl and Minikube on the instance.
3. Added the `jenkins` user to the `docker` group so pipeline jobs can run Docker commands.
4. Installed plugins: Pipeline, Git, GitHub, Docker Pipeline, Credentials Binding.
5. Added Docker Hub credentials in Jenkins (Manage Jenkins → Credentials) with ID `dockerhub-creds`.
6. Created a Pipeline job using "Pipeline script from SCM" pointing to the GitHub repo.

6. Pushing the Image

The Push Image stage uses `withCredentials` to read the Docker Hub username and password from Jenkins, so no secrets are stored in the repo. Images are tagged with the Jenkins `BUILD_NUMBER` (e.g. `1`, `2`) plus `latest`.

7. Kubernetes Deployment and Access

- `deployment.yaml` runs 2 replicas of the image pulled from Docker Hub (container port 80); the Jenkins pipeline replaces `IMAGE_TAG` with the build number.
- `service.yaml` exposes the app as a NodePort service (`smart-placement-service`):

port 80, targetPort 80, nodePort 30080.

- Verify with:

```
kubectl get pods,deployment,svc
minikube service smart-placement-service --url
```

- The URL is opened in the browser (or forwarded with `kubectl port-forward` if accessing from outside the EC2 instance).

8. Second Run (Code Change)

I changed the browser tab title to **"Smart Placement Portal - v2"**, committed and pushed to GitHub, and ran the pipeline again. Build #2 succeeded, Docker Hub showed tags `1`, `2` and `latest`, and the app in the browser displayed the new title, confirming the new version was deployed.

9. Screenshots

1. Jenkins Stage View, build #1 green
2. Application in the browser (v1)
3. `kubectl get pods,deployment,svc` output
4. Docker Hub tags
5. Jenkins credentials list
6. EC2 instance page
7. Build #2 green, v2 in the tab, Docker Hub with tags `1`, `2`, `latest`

10. Challenges and Fixes

- Minikube had stopped after the server was restarted, so the app was unreachable (connection refused). Fixed by running `minikube start` and re-exposing the service.
- `kubectl` pointed to `localhost:8080` (the Jenkins port) while the cluster was down. It worked again once Minikube was running.
- The EC2 server has limited memory (about 3.8 GB), so Minikube takes a few minutes to start.